



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



Hades PyPI Attack: 19 Packages Poisoned to Auto-Run Bun Credential Stealer

Malware Analysis Report

Classification	TLP:CLEAR
Published	2026-06-16
Version	1.0
Author	Lost Boys Cyber
Report Type	Malware Analysis Report

TLP:CLEAR | Lost Boys Cyber | Hades PyPI Attack: 19 Packages Poisoned to Auto-Run Bun Credential Stealer - Malware Analysis Report

Published: 2026-06-16 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Sample Metadata
3. Source Review and Confidence
4. Infection Chain Analysis
5. Technical Analysis
6. Indicators of Compromise
7. MITRE ATT&CK Mapping
8. Detection Engineering
9. Threat Hunting
10. Recommendations
11. References

Hades PyPI Attack: 19 Packages Poisoned to Auto-Run Bun Credential Stealer

1. Executive Summary

The Hades PyPI wave is a software supply-chain malware campaign in the Mini Shai-Hulud / Miasma lineage. Public reporting from The Hacker News, Socket, StepSecurity, Snyk, and related supply-chain security vendors describes 37 malicious Python wheel artifacts across 19 PyPI packages that abused Python startup behavior to execute a Bun-powered JavaScript credential stealer. Lost Boys Cyber assesses this as a developer-workstation and CI/CD compromise risk rather than a commodity end-user malware event: the payload is designed to steal software-delivery secrets, compromise repositories, and propagate through trusted package and source-control channels.

The directly reported execution primitive is a `*-setup.pth` file processed by Python's `site` module during interpreter startup. That means a victim does not need to explicitly import the poisoned package after installation; normal Python startup can become the execution edge. A related Hades cluster affecting computational biology and bioinformatics packages reportedly used an obfuscated one-line `__init__.py` import hook instead, but converged on the same objective: download or run the Bun runtime, execute an obfuscated JavaScript payload such as `_index.js`, and harvest credentials from developer and automation environments.

The highest-value defensive implication is credential scoping. Any organization that installed the named package versions, cached the wheels in an internal artifact repository, or ran CI jobs from affected environments should treat this as a potential broader compromise lead. Response should prioritize package inventory, endpoint and runner telemetry review, GitHub/GitLab token revocation, PyPI/npm/RubyGems/JFrog credential rotation, SSH key review, repository audit, and checks for suspicious GitHub repositories, commits, services, and workspace backdoors.

Key Finding	Defensive Implication	Confidence
37 malicious PyPI wheel artifacts were reported across 19 packages.	Inventory package managers, caches, internal mirrors, CI logs, and lockfiles for affected names and versions.	High
Startup hooks can execute without explicit package import.	Installation and interpreter-start telemetry matter; do not scope only to application runtime imports.	High
Bun runtime is used to run obfuscated JavaScript stealer logic.	Hunt for Python-to-Bun process chains, Bun downloads from GitHub, and JavaScript payloads in Python package paths.	High
Targeted secrets include package registries, cloud providers, source control, CI/CD, Kubernetes, SSH, <code>.env</code> , and AI/MCP configuration.	Treat confirmed execution as identity and supply-chain compromise; rotate tokens and audit repository actions.	High
Reported prompt-injection text attempts to mislead LLM-based package analysis tools.	Do not rely on AI-only package review without deterministic file, hook, and behavior checks.	Medium-High

2. Sample Metadata

No malware sample was detonated by Lost Boys Cyber for this intake. The report is source-backed and focuses on externally reported artifacts, behaviors, and defender detections. Hashes should be populated if a client supplies cached wheel files or endpoint-preserved payloads.

Field	Value
-------	-------

Campaign / Family	Hades, assessed as a PyPI branch of the Mini Shai-Hulud / Miasma supply-chain lineage
Malware Type	Credential stealer, supply-chain worm / propagation enabler, CI/CD secret harvester
Platform	Python package ecosystem; Linux, macOS, and Windows developer or CI hosts are in scope based on reported cross-platform scraping additions
Primary Sample SHA256	Not available from reviewed public reporting; collect from local wheel cache or PyPI mirror if exposed
File Type / Format	Python wheel artifacts, `*-setup.pth` startup hook, Python `__init__.py` import hook variants, obfuscated JavaScript payload, Bun runtime ZIP
Delivery Theme	Trojanized PyPI package releases and package-update trust abuse
First Seen / Reported	Publicly reported 2026-06-09; Socket reportedly observed the PyPI wave on 2026-06-07
Analyst Confidence	High for reported infection chain and package/version list; Medium for unverified local execution details because no internal sample detonation was performed
Primary Objective	Credential theft, source-control compromise, package propagation, CI/CD secret extraction, and potential destructive retaliation on token revocation

Affected core package/version set reported by The Hacker News and attributed to Socket:

Package	Reported Malicious Versions	Notes
bramin	0.0.2, 0.0.3, 0.0.4	PyPI wheel set associated with Hades wave
cmd2func	0.2.2, 0.2.3	PyPI wheel set associated with Hades wave
coolbox	0.4.1, 0.4.2	PyPI wheel set associated with Hades wave
dynamo-release	1.5.4	PyPI wheel set associated with Hades wave
executor-engine	0.3.4, 0.3.5	PyPI wheel set associated with Hades wave
executor-http	0.1.3, 0.1.4	PyPI wheel set associated with Hades wave
funcdesc	0.2.2, 0.2.3	PyPI wheel set associated with Hades wave
magique	0.6.8, 0.6.9	PyPI wheel set associated with Hades wave
magique-ai	0.4.4, 0.4.5	PyPI wheel set associated with Hades wave
mrbios	0.1.1, 0.1.2	PyPI wheel set associated with Hades wave
napari-ufish	0.0.2, 0.0.3	PyPI wheel set associated with Hades wave
nucbox	0.1.2, 0.1.3	PyPI wheel set associated with Hades wave

okite	0.0.7, 0.0.8	PyPI wheel set associated with Hades wave
pantheon-agents	0.6.1, 0.6.2	PyPI wheel set associated with Hades wave
pantheon-toolsets	0.5.5, 0.5.6	PyPI wheel set associated with Hades wave
spateo-release	1.1.2	PyPI wheel set associated with Hades wave
synago	0.1.1, 0.1.2	PyPI wheel set associated with Hades wave
ufish	0.1.2, 0.1.3	PyPI wheel set associated with Hades wave
uprobe	0.1.3, 0.1.4	PyPI wheel set associated with Hades wave

Related Hades/Miasma bioinformatics cluster reported in the same source stream:

Package	Reported Version(s)	Reported Entry Point
embiggen	0.11.97	Obfuscated `__init__.py` import hook
ensmallen	0.8.101	Obfuscated `__init__.py` import hook
gpsea	0.9.14	Obfuscated `__init__.py` import hook
mflux-streamlit	0.0.3, 0.0.4	Obfuscated `__init__.py` import hook
nhmpy	2.4.7	Obfuscated `__init__.py` import hook
ppkt2synergy	0.1.1	Obfuscated `__init__.py` import hook
pyphetools	0.9.120	Obfuscated `__init__.py` import hook

3. Source Review and Confidence

Source	Contribution	Confidence
The Hacker News, 2026-06-09	Consolidated public summary, package/version list, payload behavior, and campaign context.	Medium-High
Socket research, referenced by THN and search-indexed public summary	Primary technical assessment for 37 wheels, `*-setup.pth` hook, Bun-powered stealer, and Hades lineage.	High for cited claims; source page should be preserved during refresh
StepSecurity, referenced by THN	Related bioinformatics package cluster, Bun runtime rationale, AI-scanner prompt-injection detail, and gpt-pilot branch-compromise context.	Medium-High
Snyk / Semgrep / Cloudsmith context, referenced by THN	Broader lineage framing for Shai-Hulud / Miasma waves and supply-chain trust implications.	Medium
Lost Boys Cyber internal analysis	Source review, detection/hunting derivation, IOC normalization, and	Medium-High

	defensive recommendations.	
--	----------------------------	--

Directly observed from reviewed reporting: package names and versions, the `\*-setup.pth` execution primitive, Bun runtime staging, `\_index.js` payload naming, credential classes targeted, campaign markers, GitHub keyword-based staging, AI prompt-injection attempt, `gh-token-monitor` service name, and reported destructive commands. High-confidence assessment: Hades is not an isolated Python stealer; it is a branch of a recurring supply-chain playbook that abuses developer trust, CI/CD secrets, and package publication workflows. Lower-confidence items: exact final exfiltration repository names, file hashes, and per-package payload hashes were not available in the intake source and require local artifact collection or direct vendor IOC refresh.

4. Infection Chain Analysis

Phase	Reported Behavior	Defensive Collection
Package acquisition	Developer or CI installs one of the named malicious PyPI package versions, or an internal cache/mirror serves the poisoned wheel.	Package lockfiles, pip install logs, artifact proxy logs, wheel-cache directories, CI job logs.
Startup hook execution	`*-setup.pth` is processed by Python's `site` module during interpreter startup, creating execution without explicit package import.	EDR process telemetry, Python command-line records, file creation events in `site-packages`, wheel contents.
Runtime bootstrap	Payload downloads or runs Bun from GitHub and executes obfuscated JavaScript such as `_index.js`.	Network logs to GitHub release assets, archive extraction events, `bun` child processes spawned by Python.
Credential harvesting	Stealer searches developer and CI locations for GitHub, npm, PyPI, RubyGems, JFrog, CircleCI, Anthropic, AWS, GCP, Azure, Kubernetes, Docker, Vault, SSH, shell history, `.env`, `.npmrc`, `.pypirc`, Claude, and MCP secrets.	File access telemetry, process command-lines, DLP/egress telemetry, secret scanning, identity audit logs.
Exfiltration and propagation	Campaign uses GitHub-centric exfiltration and propagation logic; reported markers include Hades repository descriptions and GitHub commit keyword lookups.	GitHub audit logs, suspicious repository creation, commit search, token use from new IPs/agents, branch protection bypass attempts.
Persistence / retaliation	Reported background service `gh-token-monitor` can delete user data if a stolen GitHub token is revoked.	Service manager logs, launch agents, systemd user services, EDR file-deletion spikes, token revocation timelines.

The infection chain is especially dangerous in build systems because installation alone can create an execution edge. Traditional application security reviews that check whether a package is imported by production code are insufficient; defenders must assess whether the package was installed or resolved in any developer workstation, ephemeral build runner, notebook environment, research workstation, or internal packaging cache.

5.1 Static Analysis

Static Signal	Evidence	Analyst Value
`*-setup.pth` file	Public reporting states compromised releases shipped a startup hook processed during Python startup.	Strong detection anchor in wheel contents and installed `site-packages`; low prevalence for legitimate package-specific setup files.
`_index.js` payload	Socket-cited reporting identifies an	Provides filename and language pivot for

	obfuscated JavaScript payload named <code>`_index.js`</code> .	file and process hunts.
Bun runtime dependency	Malware downloads and runs Bun, likely as a standalone ZIP from GitHub.	Explains execution on hosts without Node.js and creates network/process indicators.
Russian-locale check	Reporting states payload checks whether the system corresponds to a Russian locale before running the stealer.	Evasion clue; absence of behavior in some sandboxes may be locale-driven.
AI-scanner prompt injection	StepSecurity-cited reporting notes plain-text prompt injection to induce LLM package analyzers to classify the package as safe.	Requires deterministic scanners to ignore natural-language safety claims in package files.
GitHub keyword lookups	Reported keywords include <code>`TheBeautifulSnadsOfTime`</code> and <code>`firedalazer`</code> .	Hunttable strings in process/network/proxy logs and repository search.
Campaign repository descriptions	<code>`Hades - The End for the Damned`</code> and <code>`Hades * The End for the Damned`</code> .	Useful for GitHub audit and open-source intelligence pivoting.

5.2 Code and Capability Analysis

The reported payload architecture follows the same strategic pattern as prior Mini Shai-Hulud and Miasma waves: obtain early execution inside a trusted package workflow, bootstrap a JavaScript runtime, run an obfuscated stealer, and use stolen credentials to access repositories or publish additional malicious artifacts. The Hades wave changes the Python execution edge from conventional import-time behavior to Python startup processing, increasing the chance that the payload runs in development tools, notebooks, and CI jobs that start Python after installation.

Capability	Reported Evidence	Operational Impact
Python startup execution	<code>`*-setup.pth`</code> processed by the <code>`site`</code> module.	Executes before normal package use and may evade import-based package reviews.
Runtime staging	Bun runtime downloaded from GitHub and used to run JavaScript.	Cross-platform execution path and reduced dependence on preinstalled Node.js.
Credential theft	Targets include GitHub, package registries, cloud credentials, Kubernetes, Docker, Vault, SSH, shell histories, <code>`.env`</code> , and AI/MCP config.	Enables repository takeover, package publishing, cloud intrusion, and lateral movement.
CI/CD memory scraping	Hades reportedly adds macOS and Windows memory scrapers beyond prior Linux <code>`/proc/{pid}/mem`</code> runner extraction.	Expands runner-secret exposure across heterogeneous build fleets.
Propagation	Reported ability to push trojanized PyPI packages using abused OIDC trust and compromised credentials.	Converts one host compromise into ecosystem and partner risk.
Workspace backdooring	Targets AI assistants and IDE workflows by modifying local workspaces.	Creates delayed execution and analyst/developer reinfection risk.

Wiper-like retaliation	`gh-token-monitor` service reportedly deletes user data if a stolen GitHub token is revoked.	Raises impact from theft to destructive behavior; token rotation should be paired with host containment.
------------------------	--	--

5.3 Dynamic Analysis Status

Lost Boys Cyber did not execute recovered Hades wheel files for this intake. The dynamic sequence below is a safe analyst reconstruction from public technical reporting and should be validated against local telemetry if affected artifacts are found.

Expected Telemetry	Why It Matters	Collection Notes
`python` or `python3` starting after a suspicious PyPI install	Trigger point for `.pth` execution.	Correlate with pip logs and package installation timestamps.
`python` spawning shell, archive tooling, curl/wget, PowerShell, or `bun`	Runtime download and payload execution.	Use process lineage, command line, and parent-child relationships.
Network requests to GitHub release assets or commit/search APIs shortly after package install	Bun and payload staging / configuration retrieval.	Proxy, DNS, TLS SNI, EDR network events, GitHub enterprise logs.
Reads of `.env`, `.npmrc`, `.pypirc`, SSH keys, Docker config, cloud credential paths, and CI variables	Credential harvesting.	Endpoint file access telemetry and DLP are useful if enabled.
GitHub repository creation, unusual commits, or repository descriptions containing Hades markers	Exfiltration and propagation.	GitHub audit log, enterprise repository inventory, token usage logs.
Creation of a `gh-token-monitor` service or launch agent	Persistence / destructive fallback.	Linux systemd user units, macOS LaunchAgents, Windows services/tasks.

6. Indicators of Compromise

Machine-readable indicators are also preserved in `iocs.csv`. The indicators below are behavior- and artifact-oriented because public hash coverage was not present in the reviewed intake.

IOC Type	Value / Pattern	Source	Operational Notes
PyPI package versions	19 package/version combinations listed in Section 2	THN / Socket	High-confidence exposure indicator; query lockfiles and caches.
File pattern	`*-setup.pth` in affected package wheels or `site-packages`	THN / Socket	Strong static detection anchor.
Payload filename	`_index.js`	THN / Socket	Hunt in Python package directories and temp locations.
Runtime	`bun` / Bun runtime ZIP from GitHub	THN / StepSecurity	Correlate with Python parent and package install timing.
Service name	`gh-token-monitor`	THN	Treat as high-severity persistence / retaliation signal.
GitHub keyword	`TheBeautifulSnadsOfTime`	THN	Reported commit-search

			payload retrieval marker.
GitHub keyword	`firedalazer`	THN	Reported Python dropper retrieval marker.
Repository description	`Hades - The End for the Damned`	THN / Socket	Audit GitHub repositories and exfil repositories.
Repository description	`Hades * The End for the Damned`	THN / Socket	Audit GitHub repositories and exfil repositories.
Destructive command fragment	`rm -rf ~/; rm -rf ~/Documents`	THN	Reported wiper behavior if token revoked.
Credential paths	`.env`, `.npmrc`, `.pypirc`, Docker config, Vault tokens, SSH keys, shell histories, Claude/MCP configs	THN / Socket	Use as behavior indicators, not standalone compromise proof.

7. MITRE ATT&CK Mapping

Technique	Name	Evidence
T1195.001	Supply Chain Compromise: Compromise Software Dependencies and Development Tools	Trojanized PyPI wheel releases delivered malware through trusted package channels.
T1059.006	Command and Scripting Interpreter: Python	`.pth` startup hook and `__init__.py` import hook execute Python-side bootstrap logic.
T1059.007	Command and Scripting Interpreter: JavaScript	Obfuscated JavaScript payload runs under Bun.
T1105	Ingress Tool Transfer	Runtime and payload components are downloaded, including Bun from GitHub.
T1552.001	Unsecured Credentials: Credentials in Files	Targeted `.env`, `.npmrc`, `.pypirc`, Docker, Vault, SSH, cloud, Claude, and MCP configuration locations.
T1552.004	Unsecured Credentials: Private Keys	Reported theft scope includes SSH keys.
T1528	Steal Application Access Token	GitHub, package registry, cloud, and CI/CD tokens are targeted.
T1005	Data from Local System	Stealer harvests local developer files and configuration.
T1053 / T1543	Scheduled Task/Job or Create/Modify System Process	`gh-token-monitor` background service indicates persistence/service creation depending on OS implementation.
T1485	Data Destruction	Reported destructive fallback removes home and Documents data if token is revoked.
T1027	Obfuscated Files or Information	JavaScript payload and import hooks are

		described as obfuscated.
T1497.003	Virtualization/Sandbox Evasion: Time Based or Environmental Checks	Russian-locale check can gate execution and affect sandbox observations.
T1567.001	Exfiltration Over Web Service: Exfiltration to Code Repository	GitHub-centric exfiltration and repository markers are reported.

8.1 Detection Logic Summary

Signal	Telemetry Source	Analytic Goal	Tuning Notes
Python package directory contains `*-setup.pth` with code execution or network logic	File integrity, EDR file events, package scanner	Catch poisoned wheel install before execution.	Some `.pth` files are legitimate; tune to affected packages and suspicious content.
Python spawns Bun shortly after pip install	EDR process telemetry	Detect runtime handoff from Python startup to Bun payload.	Developer hosts may legitimately use Bun; require Python parent and install-time proximity.
Bun downloaded from GitHub by Python-driven process chain	Proxy, DNS, EDR network	Identify runtime staging.	GitHub is common; correlate with process tree, package path, and archive extraction.
Access to multiple secret files after package install	EDR file access, DLP	Surface credential harvesting.	Requires endpoint sensor coverage; prioritize high-signal paths and burst behavior.
`gh-token-monitor` service creation	Windows Event Logs, systemd audit, launchd logs	Detect persistence or destructive monitor.	Service name is high-signal; verify exact OS implementation.
GitHub commit/repository keyword lookups	Proxy, GitHub audit, code search	Find staging/exfil markers.	Keyword strings are strong but may be absent in variants.

8.2 Sigma: Python Startup Hook to Bun Runtime

```

title: Suspicious Python Package Startup Hook Launches Bun Runtime
id: 2ad4ac90-81e9-49e9-a5ee-8c5a911f9355
status: experimental
description: Detects Python-driven execution of Bun shortly after package installation or from site-packages paths, consistent with Hades/Miasma PyPI startup-hook behavior.
author: Lost Boys Cyber
date: 2026-06-16
references:
  - https://thehackernews.com/2026/06/hades-pypi-attack-19-packages-poisoned.html
logsource:
  category: process_creation
  product: windows
detection:
  selection_bun:
    Image|endswith:
      - '\\bun.exe'
      - '\\bunx.exe'
    ParentImage|endswith:
      - '\\python.exe'
      - '\\python3.exe'
      - '\\pip.exe'
      - '\\pip3.exe'
  selection_pkg_path:

```

```

CommandLine|contains:
  - 'site-packages'
  - '.pth'
  - '_index.js'
  - 'setup.pth'
condition: selection_bun or selection_pkg_path
falsepositives:
  - Python projects that intentionally invoke Bun for frontend build tasks; tune by repository and
    parent process.
level: high
tags:
  - attack.t1195.001
  - attack.t1059.006
  - attack.t1059.007
  - attack.t1105

```

8.3 YARA: Hades/Miasma PyPI Hook and Marker Strings

```

rule LBC_Hades_Miasma_PyPI_Hook_Markers
{
  meta:
    description = "Detects reported Hades/Miasma PyPI startup-hook and campaign marker strings in
package artifacts"
    author = "Lost Boys Cyber"
    date = "2026-06-16"
    reference = "https://thehackernews.com/2026/06/hades-pypi-attack-19-packages-poisoned.html"
    tlp = "CLEAR"
  strings:
    $pth = "-setup.pth" ascii nocase
    $payload = "_index.js" ascii nocase
    $bun1 = "bun" ascii nocase
    $repo1 = "Hades - The End for the Damned" ascii
    $repo2 = "Hades * The End for the Damned" ascii
    $kw1 = "TheBeautifulSnadsOfTime" ascii
    $kw2 = "firedalazer" ascii
    $svc = "gh-token-monitor" ascii
    $wipe1 = "rm -rf ~/" ascii
    $wipe2 = "rm -rf ~/Documents" ascii
  condition:
    uint16(0) == 0x4b50 or filesize < 25MB and (
      ($pth and $payload and $bun1) or
      any of ($repo*) or
      any of ($kw*) or
      $svc or
      all of ($wipe*)
    )
}

```

8.4 KQL: Microsoft Defender for Endpoint Hunt

```

// Hades/Miasma-style Python-to-Bun execution after package installation
let lookback = 30d;
DeviceProcessEvents
| where Timestamp > ago(lookback)
| where FileName in~ ("bun.exe", "bun", "bunx.exe", "bunx")
  or ProcessCommandLine has_any ("_index.js", "setup.pth", "site-packages")
| where InitiatingProcessFileName in~ ("python.exe", "python3.exe", "pip.exe", "pip3.exe", "sh",
"bash", "powershell.exe", "pwsh.exe", "cmd.exe")
  or InitiatingProcessCommandLine has_any ("pip install", "site-packages", ".pth")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, InitiatingProcessFileName,
InitiatingProcessCommandLine, ReportId
| order by Timestamp desc

// Secret-file access burst following suspicious Python package activity
let suspiciousDevices = DeviceProcessEvents
| where Timestamp > ago(30d)
| where ProcessCommandLine has_any ("bramin", "cmd2func", "coolbox", "dynamo-release", "executor-
engine", "executor-http", "funcdesc", "magique", "magique-ai", "mrbios", "napari-ufish", "nucbox",

```

```
"okite", "pantheon-agents", "pantheon-toolsets", "spateo-release", "synago", "ufish", "uprobe",
"_index.js", "setup.pth")
| summarize by DeviceId;
DeviceFileEvents
| where Timestamp > ago(30d)
| where DeviceId in (suspiciousDevices)
| where FolderPath has_any (".env", ".npmrc", ".pypirc", ".ssh", "config.json", "credentials", "kube",
"claude", "mcp", "vault", "docker")
| project Timestamp, DeviceName, InitiatingProcessFileName, InitiatingProcessCommandLine, ActionType,
FolderPath, SHA256
| order by Timestamp desc
```

8.5 SPL: Splunk Endpoint Hunt

```
index=endpoint (process_name=bun OR process_name=bun.exe OR process="*_index.js*" OR
process="*setup.pth*" OR process="*site-packages*")
| search parent_process_name IN (python, python3, python.exe, pip, pip3, pip.exe, bash, sh, cmd.exe,
powershell.exe, pwsh.exe)
| table _time host user parent_process_name parent_process process_name process process_path
| sort - _time
```

8.6 Validation and Blind Spots

These analytics are source-derived and should be validated in a lab by installing a benign test wheel containing a `.pth` startup hook that spawns a harmless child process, then replaying expected telemetry. Blind spots include short-lived ephemeral CI runners without EDR, network egress that only records destination domains, developer machines where Bun is already common, and variants that rename `\_index.js` or avoid GitHub keyword markers. Detections should be paired with package inventory and identity audit logs, not treated as a standalone binary verdict.

9.1 Hunt Hypotheses

- A developer workstation or CI runner installed one of the affected Hades package versions and subsequently executed Python, causing `.pth` startup-hook execution.
- The Hades payload downloaded Bun from GitHub or invoked an existing Bun runtime to execute an obfuscated JavaScript stealer.
- Stolen GitHub or registry credentials were used to create repositories, publish packages, change workflows, or access secrets from unexpected IP addresses or user agents.
- The destructive `gh-token-monitor` service was installed on a host where a GitHub token was later revoked.

9.2 Package Exposure Hunt

```
// Search endpoint command lines for installation or reference to affected Hades PyPI packages
let affected = dynamic(["bramin", "cmd2func", "coolbox", "dynamo-release", "executor-engine", "executor-http", "funcdesc", "magique", "magique-ai", "mrbios", "napari-ufish", "nucbox", "okite", "pantheon-agents", "pantheon-toolsets", "spateo-release", "synago", "ufish", "uprobe"]);
DeviceProcessEvents
| where Timestamp > ago(60d)
| where ProcessCommandLine has_any (affected)
| where ProcessCommandLine has_any ("pip install", "pip3 install", "uv pip install", "poetry add")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, InitiatingProcessFileName, InitiatingProcessCommandLine
| order by Timestamp desc
```

9.3 GitHub Marker Hunt

```
// Hunt proxy/network telemetry for reported GitHub keyword and repository-description markers
DeviceNetworkEvents
| where Timestamp > ago(60d)
| where RemoteUrl has_any ("github.com", "api.github.com", "raw.githubusercontent.com")
| where InitiatingProcessCommandLine has_any ("TheBeautifulSnadsOfTime", "firedalazer", "Hades - The End for the Damned", "Hades * The End for the Damned")
| where AdditionalFields has_any ("TheBeautifulSnadsOfTime", "firedalazer", "Hades - The End for the Damned", "Hades * The End for the Damned")
| project Timestamp, DeviceName, InitiatingProcessFileName, InitiatingProcessCommandLine, RemoteUrl, RemoteIP, ReportId
| order by Timestamp desc
```

9.4 Persistence / Retaliation Hunt

```
// Hunt for the reported gh-token-monitor service and destructive command fragments
DeviceProcessEvents
| where Timestamp > ago(60d)
| where ProcessCommandLine has_any ("gh-token-monitor", "rm -rf ~/", "rm -rf ~/Documents")
   or FileName has_any ("systemctl", "launchctl", "sc.exe", "schtasks.exe") and ProcessCommandLine has
"gh-token-monitor"
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, InitiatingProcessFileName,
InitiatingProcessCommandLine
| order by Timestamp desc
```

9.5 Triage Guidance for Hunt Hits

A meaningful hit is any affected package/version installation correlated with `.pth` files, _index.js`, Bun execution, suspicious GitHub API access, or secret-file access. Escalate immediately if a hit occurs on a CI runner, release engineer workstation, package maintainer host, or host with cloud deployment permissions. Benign lookalikes include legitimate Python projects using Bun for frontend builds and legitimate .pth` files from packaging tools; tune by requiring affected package names, suspicious file paths, or temporal proximity to installation.`

10. Recommendations

Priority	Owner	Action	Rationale
Critical	DevOps / Platform	Inventory package lockfiles, dependency manifests, internal PyPI mirrors, CI caches, and workstation wheel caches for affected packages and versions.	Confirms exposure even if endpoint telemetry aged out.
Critical	Identity / AppSec	Rotate GitHub, PyPI, npm, RubyGems, JFrog, CircleCI, cloud, Vault, Kubernetes, SSH, Anthropic, Claude, and MCP credentials from exposed hosts.	The payload targets broad developer and automation secrets.
Critical	SecOps	Isolate hosts with confirmed execution before token revocation where feasible.	Reported <code>gh-token-monitor` behavior can trigger destructive deletion after token revocation.</code>
High	GitHub / SCM Admins	Review repository creation, commit history, workflow edits, Actions secrets access, package publication, and token usage around exposure windows.	Hades uses GitHub-centric exfiltration and propagation logic.
High	Endpoint Engineering	Deploy detections for Python-to-Bun process chains, suspicious <code>.pth` files, <code>_index.js`</code>, and service creation.</code>	Provides immediate behavioral coverage for current and related waves.
High	Build Engineering	Restrict network egress from build runners, pin dependencies by hash, and block package install hooks where workflows allow.	Reduces blast radius of install-time supply-chain malware.
Medium	AppSec	Add deterministic package scanning for <code>.pth` files,</code>	AI-only package review can be manipulated by prompt-

		obfuscated import hooks, and prompt-injection text in source packages.	injection content.
Medium	IR / Forensics	Preserve suspicious wheels, package caches, shell histories, EDR timelines, and CI runner disk images before cleanup.	Enables hash extraction and customer-specific IOC enrichment.
Medium	Governance	Require maintainer MFA, scoped publish tokens, OIDC trust review, and protected release workflows.	Limits propagation through compromised maintainer or CI identities.

Immediate containment sequence for confirmed exposure:

- 1. Snapshot the host or runner if feasible; preserve `site-packages`, pip/uv/poetry caches, shell history, CI job logs, and EDR timelines.
- 2. Disconnect or quarantine affected hosts from sensitive networks.
- 3. Revoke and rotate high-value credentials, prioritizing GitHub, package registries, cloud providers, Kubernetes, Vault, and SSH.
- 4. Audit repositories and package publications for suspicious commits, releases, workflows, and exfiltration repositories.
- 5. Remove affected packages, clear caches, rebuild from trusted lockfiles, and redeploy runners from known-good images.
- 6. Run the detection and hunting queries in this report across the previous 60 days, extending lookback if artifact mirrors retain older packages.

11. References

- [1] The Hacker News, "Hades PyPI Attack: 19 Packages Poisoned to Auto-Run Bun Credential Stealer," published 2026-06-09. <https://thehackernews.com/2026/06/hades-pypi-attack-19-packages-poisoned.html>
- [2] Socket, "Shai-Hulud Descends to Hades: Miasma Worm Campaign Spreads with New PyPI Wave," referenced by The Hacker News and indexed public summaries. <https://socket.dev/blog/shai-hulud-descends-to-hades-miasma-pypi-wave>
- [3] StepSecurity, "The Hades Campaign PyPI Packages," referenced by The Hacker News for bioinformatics package cluster, Bun runtime detail, and AI-scanner prompt-injection behavior. <https://www.stepsecurity.io/blog/the-hades-campaign-pypi-packages>
- [4] StepSecurity, "Pythagora-io/gpt-pilot compromised on GitHub; Shai-Hulud credential stealer blocked by Python linter," referenced by The Hacker News for related lineage context. <https://www.stepsecurity.io/blog/pythagora-io-gpt-pilot-compromised-on-github-shai-hulud-credential-stealer-blocked-by-python-linter>
- [5] Snyk, "node-gyp supply-chain compromise / self-propagating npm worm," referenced by The Hacker News for Shai-Hulud / Miasma lineage context. <https://snyk.io/blog/node-gyp-supply-chain-compromise-self-propagating-npm-worm-binding-gyp/>
- [6] Semgrep, "Miasma v2 self-spreading npm worm now uses malicious binding.gyp file," referenced by The Hacker News for recurring Bun-runtime stealer behavior. <https://semgrep.dev/blog/2026/miasma-v2-self-spreading-npm-worm-now-uses-malicious-bindinggyp-file-and-compromises-57-packages/>
- [7] Cloudsmith, "Miasma worm's path of destruction," referenced by The Hacker News for package-repository trust implications. <https://cloudsmith.com/blog/miasma-worms-path-of-destruction>